

Guide : pull requests, mise en production et déploiement

Deux dépôts font vivre le site de l'agence OPTINOV :

Dépôt	Rôle	Hébergement	Adresse
christ544/optinov-tableau	le tableau de bord (Payload CMS)	Render	https://optinov-dashboard.onrender.com/admin
christ544/Optinov-agence	le site vitrine (Next.js, statique)	Cloudflare	https://optinov-agence.christkangah14.workers.dev

Tu es propriétaire des deux. Paul-Aymard propose les changements par **pull request** (PR) ; c'est toi qui relis et qui fusionnes. Ce guide explique comment relire, tester, fusionner, proposer toi-même un changement, et comment déployer sur PlanetHoster le jour où l'hébergement change.

1. Le principe

- Le travail est découpé en **étapes**. Chaque étape vit sur sa propre branche et donne **une PR vers main**.
- Les branches du tableau de bord sont **empilées** : l'étape 2 contient l'étape 1, l'étape 3 contient les étapes 1 et 2, etc. Il faut donc **fusionner dans l'ordre**.
- Tant que la PR #1 n'est pas fusionnée, l'onglet « Files changed » de la PR #2 montre aussi les fichiers de l'étape 1. C'est normal : il se réduit tout seul dès que #1 est fusionnée. Pour relire, ne regarde que ce que la description de la PR annonce.

PR	Dépôt	Branche	Contenu
#1	tableau de bord	etape-1-environnement-local	Cloudinary facultatif, images sur disque en local
#2	tableau de bord	etape-2-nettoyage	retrait du gabarit Cloudflare, ESLint réparé, README en français
#3	tableau de bord	etape-3-habillage-optinov	charte OPTINOV dans l'admin, accueil sur mesure, logo
#4	tableau de bord	etape-4-securite	rôles administrateur / éditeur, CORS et CSRF, uploads, GraphQL coupé ; contient une migration
#5	tableau de bord	etape-5-contenus	paramètres du site, réalisations, témoignages, équipe, FAQ ; contient deux migrations
site 1	site vitrine	dashboard-payload	le site lit le tableau de bord, seule source de vérité ; retrait des anciens outils d'édition
#6	tableau de bord	etape-6-demandes	rubrique Demandes, alertes e-mail par le Gmail de l'agence ; contient une migration
#7	tableau de bord	etape-7-planethoster	déploiement sur PlanetHoster : fichier de démarrage, script de déploiement, guide
site 2	site vitrine	formulaires-vers-dashboard	les formulaires envoient les demandes au tableau de bord

Ordre à respecter : #1, #2, #3, #4, #5, puis la PR « site 1 » **après** que le tableau de bord a été redéployé (section 5) ; ensuite #6 et #7, puis « site 2 » après le redéploiement suivant. Dans l'autre sens, la construction du site échoue exprès, parce que les nouvelles rubriques n'existent pas encore dans l'API.

Raccourci : tout fusionner d'un coup. Les branches étant empilées, celle de l'étape 7 contient déjà les six précédentes. Fusionner la **seule PR #7** met donc `main` à jour en une fois, et GitHub referme les six autres en « Merged » tout seul. C'est le chemin à prendre quand la relecture pas à pas n'est pas possible — en gardant la règle qui compte vraiment : le site vitrine ne se reconstruit qu'**après** que le tableau de bord est en ligne avec ses nouvelles rubriques.

Après la fusion de #6, renseigner dans Render trois variables : `SMTP_USER` (l'adresse Gmail de l'agence), `SMTP_PASS` (un mot de passe d'application Google, créé dans le compte Google > Sécurité > Validation en deux étapes > Mots de passe des applications) et `ALERTE_EMAIL` (l'adresse qui reçoit les alertes). Sans elles, les demandes sont enregistrées mais personne n'est prévenu.

2. Relire une PR (sur GitHub)

- Ouvre la PR, lis la **description** dans l'onglet « Conversation » : elle dit ce qui change et ce qui a été vérifié.
- Onglet « **Files changed** » : parcours les fichiers. Pour poser une question ou demander une correction, clique sur le **+** en face de la ligne et écris un commentaire.
- Bouton « **Review changes** » (en haut à droite) :
 - **Approve** : tout est bon ;
 - **Request changes** : tu attends une correction avant de fusionner ;

- **Comment** : simple remarque, sans bloquer.

Il n'y a pas d'intégration continue sur ces dépôts : GitHub ne lance aucun test à ta place. Le test, c'est toi qui le fais en local (section 3).

3. Tester en local avant de fusionner

Le tableau de bord

Prérequis : le projet installé selon son README (Node 22, PostgreSQL, fichier `.env`).

```
git fetch origin
git checkout etape-4-securite      # la branche de la PR à tester
npm install                       # les dépendances changent selon les étapes
npm run migrate                   # applique les migrations s'il y en a de nouvelles
npm run dev                       # puis http://localhost:3000/admin
```

Vérifications rapides, dans un second terminal :

```
npm run lint
npm run test:int
npm run test:e2e                  # la première fois : npx playwright install chromium
```

Le premier chargement de l'admin prend environ une minute : ce n'est pas un bug. Pour voir le tableau de bord rempli, `npm run seed` charge un jeu de démonstration (uniquement sur une base locale). Quand tu as fini : `git checkout main`.

Le site

Le site lit le tableau de bord au moment de sa construction. Pour le tester avec ton tableau de bord local lancé sur le port 3000 :

```
git fetch origin
git checkout dashboard-payload
npm install
DASHBOARD_URL=http://localhost:3000 npm run sync  # récupère le contenu
npm run dev                                     # puis http://localhost:3000 ... ou
3001 si le port est pris
```

Pour une construction complète, comme sur Cloudflare :

```
DASHBOARD_URL=http://localhost:3000 npm run build # produit le dossier out/
```

4. Fusionner

1. Fusionne **dans l'ordre des numéros** (#1 avant #2, #2 avant #3, etc.).
2. Clique sur la flèche du bouton vert et choisis « **Create a merge commit** ». Pas « **Squash and merge** », pas « **Rebase and merge** » : ces deux options réécrivent les commits, et GitHub croirait alors que la

PR suivante contient encore l'étape précédente (diff gonflé, faux conflits).

3. « Delete branch » après la fusion : possible, sans risque, la branche suivante contient déjà ces commits.
4. Ouvre la PR suivante et vérifie que « Files changed » ne montre plus que ses propres fichiers. Si oui, tu peux la relire et la fusionner à son tour.
5. La PR du site se fusionne **en dernier**, une fois le tableau de bord redéployé.

5. Ce qui se passe après une fusion

Tableau de bord (Render)

- **Render redéploie main automatiquement** (`autoDeploy: true` dans `render.yaml`). Une fusion dans `main` = une mise en production. Fusionne donc uniquement ce que tu as relu et testé.
- Les **migrations** de base s'appliquent toutes seules au démarrage du service. Celle de l'étape 4 déclare administrateurs tous les comptes existants ; celles de l'étape 5 créent les nouvelles rubriques et y installent les vingt questions de la FAQ et les coordonnées du site.
- Surveille le déploiement dans Render (onglet « Events » puis « Logs »). Si le build échoue, le service précédent reste en ligne : rien n'est cassé, préviens Paul-Aymard avec le message d'erreur.
- Le service gratuit s'endort après quinze minutes sans visite et met jusqu'à une minute à se réveiller. Ce n'est pas une panne.

Tableau de bord (PlanetHoster)

Rien ne part tout seul : une fusion ne fait que poser le nouveau code sur GitHub. Il faut ensuite lancer le déploiement sur le serveur (`./deployer.sh` , section 9). Tant qu'on ne l'a pas fait, la version en ligne reste l'ancienne — ce qui est aussi une sécurité : on choisit le moment.

Site (Cloudflare)

- Après la fusion de la PR du site, lance une construction dans Cloudflare (Workers & Pages, projet `optinov-agence` , « Retry deployment » ou un nouveau déploiement), ou pousse un commit.
- Ensuite, **chaque enregistrement dans le tableau de bord reconstruit le site** tout seul, deux minutes après le dernier enregistrement, grâce au Deploy Hook renseigné dans Render (`SITE_DEPLOY_HOOK`). La modification est en ligne quelques minutes plus tard.
- Si le tableau de bord est injoignable pendant la construction, celle-ci échoue volontairement : la version précédente du site reste en ligne.

Sur ton PC, après une fusion : `git checkout main && git pull` .

6. Proposer toi-même une modification

La même règle vaut pour tout le monde : **jamais de commit directement sur main** , toujours une branche et une PR. Exemple sur le tableau de bord :

```
git checkout main && git pull          # partir de la version en ligne
git checkout -b correction-libelle-faq  # une branche au nom parlant
# ... modifier les fichiers ...
git add -A
git commit -m "FAQ : corrige le libellé du thème Support"
git push -u origin correction-libelle-faq # envoie la branche sur GitHub
gh pr create --base main --fill          # ouvre la PR (ou bouton « Compare & pull
request » sur GitHub)
```

- **Compléter une PR déjà ouverte** : fais tes commits sur **la même branche** et `git push`. La PR se met à jour toute seule.
- **Une PR par sujet** : une correction, une branche. Deux sujets mélangés rendent la relecture et le retour en arrière difficiles.
- **Avant de pousser** : `npm run lint` et `npm run test:int` doivent passer.
- **Migrations** : si tu modifies une collection, `npm run migrate:create` puis commite le fichier de migration avec ton code. Sans lui, la production démarre sur un schéma incomplet.

7. Demander une correction

Commente sur la ligne concernée ou choisis « Request changes ». L'auteur pousse un commit supplémentaire sur **la même branche** : la PR se met à jour toute seule, tu n'as rien à rouvrir. Relis le nouveau commit, puis approuve et fusionne.

8. En cas de conflit

Si GitHub affiche « This branch has conflicts that must be resolved », **ne résous pas le conflit dans l'éditeur GitHub** : avec des branches empilées, cela casse les PR suivantes. Préviens l'auteur, il rebase la branche et repousse ; le conflit disparaît.

9. Déployer le tableau de bord sur PlanetHoster (NOC)

Le site vitrine reste sur Cloudflare : il est statique, et rien de ce qui suit ne le concerne. Seul le tableau de bord déménage.

Deux fichiers du dépôt n'existent que pour cet hébergement. Render ne les lit pas ; ils peuvent donc rester en place pendant toute la durée de la bascule.

Fichier	Rôle
<code>server.cjs</code>	le « fichier de démarrage » que Passenger exécute. Sans lui, l'application ne démarre pas sur NOC.
<code>deployer.sh</code>	enchaîne les gestes d'un déploiement, dans le seul ordre qui fonctionne.

A. Ce qu'on crée dans le panneau N0C

Élément	Valeur
Sous-domaine	<code>admin.<domaine de l'agence></code> , avec son certificat SSL
Base PostgreSQL	une base et un utilisateur dédiés (menu « Bases de données »)
Application Node.js	version 22 , mode production
→ Répertoire de l'application	<code>optinov-admin</code> (le clone du dépôt, voir D)
→ Fichier de démarrage	<code>server.cjs</code> — et surtout pas <code>npm start</code> , que Passenger ne sait pas exécuter
→ URL de l'application	le sous-domaine créé ci-dessus

B. Les six règles de N0C

Elles ne sont écrites nulle part chez l'hébergeur, et chacune coûte une demi-journée quand on ne les connaît pas. `deployer.sh` les applique toutes : elles sont listées ici pour comprendre ce qu'il fait, et pour les cas où l'on travaille à la main.

1. **Activer l'environnement Node avant tout** `npm` (la commande d'activation est affichée dans le panneau, en haut de l'application). Sans cela, `npm` installe avec le mauvais Node et l'application refuse de démarrer, sur des erreurs de modules natifs (`sharp`, en général) alors que l'installation s'est « bien passée ».
2. **Installer avec les dépendances de développement** : `NODE_ENV=development npm ci --include=dev`. Sans elles, TypeScript manque et le build échoue.
3. **Briquer les ressources au build**, sinon le processus est tué (`os error 11`).
4. `node_modules` **est un lien symbolique à l'exécution, un vrai dossier au build**. `npm ci` remplace le lien par un dossier : on construit, **puis** on repose le lien. Jamais l'inverse.
5. **Construire avec Webpack** (`next build --webpack`). Turbopack, le constructeur par défaut de Next 16, réclame une bibliothèque système que cet hébergement n'a pas : le build s'arrête sur « Turbopack is not supported on this platform ».
6. **Passenger garde le processus en mémoire** : après tout changement, `touch tmp/restart.txt` dans le dossier de l'application, ou ↻ dans le panneau.

C. Le fichier `.env`, sur le serveur

Sur Render, les variables se saisissent dans l'interface. Sur PlanetHoster, elles vivent dans un fichier `.env` à **la racine de l'application**, créé à partir de `.env.example` et **jamais commité** — il contient le mot de passe de la base et celui des e-mails.

Variable	Valeur
DATABASE_URI	la chaîne de connexion de la base PostgreSQL N0C
PAYLOAD_SECRET	une chaîne aléatoire : <code>openssl rand -hex 32</code> . La garder stable : en changer déconnecte tout le monde
PAYLOAD_PUBLIC_SERVER_URL	<code>https://admin.<domaine></code> : exactement l'adresse par laquelle on ouvre l'admin, sinon tout enregistrement est refusé
FRONTEND_URL	l'adresse du site vitrine (autorise ses appels à l'API)
CLOUDINARY_CLOUD_NAME , CLOUDINARY_API_KEY , CLOUDINARY_API_SECRET	à recopier depuis Render pour garder les images déjà en ligne (voir F)
SITE_DEPLOY_HOOK	l'URL du Deploy Hook Cloudflare du site vitrine
SMTP_HOST	<code>smtp.gmail.com</code>
SMTP_PORT	<code>465</code>
SMTP_USER	l'adresse Gmail de l'agence
SMTP_PASS	le mot de passe d'application Google (seize caractères), jamais le mot de passe du compte
SMTP_FROM	l'adresse affichée comme expéditeur (souvent la même que SMTP_USER)
ALERTES_EMAIL	l'adresse qui reçoit les alertes de nouvelles demandes

SMTP_HOST et SMTP_PORT étaient fixés par `render.yaml` : sur PlanetHoster, il faut les écrire soi-même. Sans eux, les demandes sont bien enregistrées mais personne n'est prévenu.

D. Premier déploiement, en SSH sur le serveur

```
# 1. Le code, dans le dossier déclaré comme répertoire de l'application
git clone https://github.com/christ544/optinov-tableau.git ~/optinov-admin
cd ~/optinov-admin

# 2. Créer l'application Node.js dans le panneau N0C (section A) en pointant sur
# ce dossier, puis revenir ici. Le panneau y installe tmp/ et le lien
# node_modules : c'est normal, et c'est justement ce qu'on veut.

# 3. Les variables (section C)
cp .env.example .env && nano .env

# 4. Le déploiement lui-même
chmod +x deployer.sh
N0C_UTILISATEUR=<votre identifiant> DOMAINE_ADMIN=https://admin.<domaine> \
./deployer.sh premier
```

Le script s'arrête de lui-même si le `.env` est incomplet, si le build échoue (en restaurant le build précédent) ou si l'application ne répond pas après le redémarrage. Il se termine par deux vérifications : la page de connexion et l'API des paramètres doivent répondre.

Pour éviter de retaper les deux variables à chaque fois, renseigner `UTILISATEUR` et `DOMAINE` en haut de `deployer.sh` — le fichier est fait pour ça.

E. Le premier compte administrateur

Ouvrir `https://admin.<domaine>/admin` : tant qu'aucun compte n'existe, Payload propose lui-même de créer le premier, qui est administrateur. **Le faire tout de suite** : cette page est ouverte à quiconque connaît l'adresse.

Si la base a été reprise de Render (voir F), les comptes existants sont déjà là et cette page n'apparaît pas.

F. Les images, et la base

Deux façons de procéder, selon ce qu'on veut garder.

- **Garder Cloudinary** (le plus simple) : recopier les trois variables `CLOUDINARY_*` depuis Render. Les images déjà en ligne continuent de s'afficher, les nouvelles partent au même endroit, et il n'y a rien à sauvegarder sur le serveur.
- **Passer au disque du serveur** : laisser les trois variables vides. Les images s'écrivent alors dans `./media`, à l'intérieur de l'application. Ce dossier survit aux déploiements (`git pull` n'y touche pas), mais c'est à vous de le sauvegarder — et les images déjà déposées sur Cloudinary ne suivent pas.

Pour la **base**, si le tableau de bord a déjà servi sur Render : exporter la base Neon (`pg_dump`) et l'importer dans la base NOC **avant** le premier déploiement. Sinon, on repart d'une base vide, et les migrations la remplissent (rubriques, vingt questions de FAQ, coordonnées du site).

G. Prévenir le site vitrine de la nouvelle adresse

Le site vitrine pointe encore sur l'adresse Render, écrite en dur comme valeur par défaut. Il faut donc, dans **Cloudflare** > Workers & Pages > `optinov-agence` > Settings > Variables d'environnement (côté **build**) :

Variable	Valeur
<code>DASHBOARD_URL</code>	<code>https://admin.<domaine></code> — utilisée à la construction, pour lire les contenus
<code>NEXT_PUBLIC_DASHBOARD_URL</code>	<code>https://admin.<domaine></code> — inscrite dans les pages, pour l'envoi des formulaires

Puis relancer une construction du site. Tant que ces deux variables ne sont pas posées, le site continue d'interroger Render : les contenus restent ceux de l'ancien tableau de bord, et les formulaires y envoient les demandes.

Vérifier ensuite, côté tableau de bord, que `FRONTEND_URL` porte bien l'adresse du site : c'est elle qui autorise les formulaires à écrire.

H. Mises à jour suivantes

Une mise à jour de **code** (pas de contenu : le contenu se fait dans l'admin et prend effet immédiatement) :

```
cd ~/optinov-admin
./deployer.sh
```

Le script récupère le code, installe, applique les migrations, construit, repose le lien `node_modules`, redémarre et vérifie que tout répond.

I. Si ça ne démarre pas

1. Lire `tmp/demarrage.log` dans le dossier de l'application. `server.cjs` y écrit son propre compte rendu — Node, port, variables présentes ou absentes, mémoire — parce que le journal de Passenger, lui, n'est pas lisible depuis le compte : il ne donne qu'une erreur 500 et un « Error ID » réservé au support.
2. `node_modules` n'est plus un lien ? Relancer `./deployer.sh`, qui le répare.
3. « Vous n'êtes pas autorisé à effectuer cette action » à l'enregistrement, alors que la consultation marche : `PAYLOAD_PUBLIC_SERVER_URL` ne correspond pas à l'adresse ouverte dans le navigateur.
4. Le journal s'arrête après « Demarrage demande », sans erreur : le processus a été tué, presque toujours pour dépassement du quota mémoire. Les lignes « preparation en cours » donnent la courbe.
5. Une variable d'environnement modifiée ne prend effet qu'après `touch tmp/restart.txt`.

10. Ce qui reste à faire après les fusions

Une fois les PR fusionnées et le tableau de bord redéployé, ces gestes terminent la mise en service. Ils ne demandent pas de code, mais ils demandent les accès de l'agence.

Où se saisissent les variables, selon l'hébergement du tableau de bord :

Hébergement	Où	Prise en compte
Render	service <code>optinov-dashboard</code> > Environment	Render redémarre tout seul
PlanetHoster	le fichier <code>.env</code> de l'application (section 9.C)	après <code>touch tmp/restart.txt</code>

Ci-dessous, « renseigner une variable » veut dire : à l'endroit indiqué par ce tableau. De même, « les journaux » désigne Render > Logs, ou `tmp/demarrage.log` et le journal Passenger sur PlanetHoster.

A. Brancher les alertes e-mail (dès la fusion de #6)

1. Dans le **compte Google de l'agence** : Sécurité > Validation en deux étapes (l'activer si besoin) > **Mots de passe des applications** > créer un mot de passe nommé « Tableau de bord ». Google affiche seize caractères : les copier.
2. Renseigner :

Variable	Valeur
<code>SMTP_USER</code>	l'adresse Gmail de l'agence
<code>SMTP_PASS</code>	les seize caractères du mot de passe d'application (jamais le mot de passe du compte)
<code>ALERTE_EMAIL</code>	l'adresse qui doit recevoir les alertes (peut être la même)

Sur Render, `SMTP_HOST` et `SMTP_PORT` sont déjà fixés par `render.yaml`. Sur PlanetHoster, il faut les écrire aussi : `smtp.gmail.com` et `465`.

1. **Tester avec un vrai formulaire** depuis le site en ligne (page Contact) : la demande doit apparaître dans le tableau de bord, rubrique Demandes ; l'alerte doit arriver sur `ALERTES_EMAIL` ; l'accusé de réception sur l'adresse saisie dans le formulaire. Ensuite, supprimer la demande de test dans le tableau de bord.
2. Si l'alerte n'arrive pas : dans les journaux, chercher « Alerte NON envoyée ». La cause est presque toujours un mot de passe d'application mal copié ou la validation en deux étapes désactivée.

B. Vérifier la reconstruction automatique du site

1. Dans **Cloudflare** > Workers & Pages > `optinov-agence` > Settings > Builds > **Deploy hooks** : créer un hook s'il n'existe pas, copier son URL.
2. Renseigner `SITE_DEPLOY_HOOK` = cette URL.
3. Test de bout en bout : modifier un témoignage dans le tableau de bord, enregistrer, attendre deux minutes (regroupement) puis le temps du build Cloudflare (deux à trois minutes). La modification doit apparaître sur le site. Si rien ne bouge : dans les journaux, chercher « Reconstruction du site » ; puis Cloudflare > Deployments, vérifier qu'un build est parti et lire son journal.

C. Créer les comptes de l'équipe

Dans le tableau de bord > Administration > Utilisateurs > Ajouter : nom, e-mail, mot de passe provisoire, rôle. **Administrateur** pour les personnes qui gèrent les comptes, **Éditeur** pour celles qui ne touchent qu'au contenu. Chaque personne change ensuite son mot de passe depuis « Mon compte ». Supprimer les comptes de test qui ne servent plus.

D. Contenus à obtenir de la direction

Ces éléments sont encore écrits « [À compléter] » dans le code du site, ou vides. Ils ne se saisissent pas dans le tableau de bord : ils demandent une modification du code du site (une PR) une fois les valeurs arrêtées.

Élément	Où dans le code du site
Raison sociale, forme juridique, capital, coordonnées de l'hébergeur	<code>app/(site)/mentions-legales/page.js</code>
Date de mise à jour, référent protection des données, durées de conservation	<code>app/(site)/politique-de-confidentialite/page.js</code>
Prix des trois offres PROS.CARDS	<code>content/prosCards.js</code>
Chiffres clés et logos clients de l'accueil	<code>content/site.js</code> (<code>chiffresCles</code> , <code>logosClients</code>)

En revanche, **coordonnées, horaires, RCCM, directeur de publication, hébergeur, réseaux sociaux et liens PROS.CARDS** se saisissent directement dans le tableau de bord, rubrique Paramètres du site, et arrivent sur le site à la reconstruction suivante.

E. Remplir le tableau de bord

Le site affiche ce que contient le tableau de bord. À saisir pour que les pages correspondantes s'affichent : au moins un article de blog, les illustrations des cinq pages Services, les témoignages, l'équipe, les réalisations (cochées « Publiée »). Tant qu'une rubrique est vide, la section du site correspondante se masque ou affiche un message d'attente.

Aide-mémoire

Je veux...	Commande ou action
voir les PR ouvertes	onglet « Pull requests » du dépôt
tester une PR	<code>git fetch origin && git checkout <branche></code> puis <code>npm install</code>
fusionner	« Create a merge commit », dans l'ordre des numéros
proposer un changement	branche + commits + <code>git push -u origin <branche></code> + PR
compléter une PR ouverte	commits sur la même branche + <code>git push</code>
revenir à la version en ligne	<code>git checkout main && git pull</code>
ce qui a changé en production	historique de <code>main</code> sur GitHub, ou <code>git log --oneline main</code>
reconstruire le site	enregistrer n'importe quoi dans le tableau de bord, ou « Retry deployment » dans Cloudflare
déployer sur PlanetHoster	<code>cd ~/optinov-admin && ./deployer.sh</code>
redémarrer sur PlanetHoster	<code>touch tmp/restart.txt</code> dans le dossier de l'application
comprendre un démarrage raté	<code>tail -40 ~/optinov-admin/tmp/demarrage.log</code>